AI 321 / EE 298: Machine Learning for Sequential Data

Machine Exercise 2

Subspace & State-Space Identification

Two system-identification problems: a Newell–Lee evaporator subspace-ID case study using CVA, MOESP, and SSARX, and an unknown continuous state-space system identified from input/output waveform data. It also includes the course's standard personalization and reflection/academic-integrity components.

Machine Exercise

September 15, 2026

Contents

2.1 Instructions **3**

2.2 The dataset is different for each student **3**

2.3 Problem 1: Subspace Identification Methods (SIM) **4**

2.4 Problem 2: Unknown State-Space Model **4**

Reflection and Academic Integrity Declaration **7**

2.1 Instructions

INSTRUCTIONS. This is an open-notes, open-books exercise to be done individually. You are to submit a lab report detailing the process and analysis of your chosen approach, and an executable Jupyter notebook or a MATLAB Live script `.mlx` file. Make sure to write your name and student number in the first part of your submission so that it is easily identifiable to you. Include figure, plots, diagrams, tables, etc. as necessary in your discussion to clarify your answers and analysis of the results.

From your `.mlx` or `.ipynb`, generate the PDF output with comments, plots, answers to the questions and discussion. Submit as a single zipped file the `.mlx` or `.ipynb` and the the PDF output. If there are other external files needed (*.csv, *.wav, etc) include everything in the zipped file and upload to the UVLe submission bin before the deadline. Anything submitted after the deadline can still be graded but with a 0.6 multiplier penalty to the final score. The submitted zipped file must be easily identifiable to you, use the suggested file name `<Surname>_AI321_ME02.zip` when submitting any machine exercise.

2.2 The dataset is different for each student

Every student works with a seed and a few parameters derived deterministically from their own official student number, see the code snippet below. This means some specifications of the exercise are unique for each student. In this exercise, each student will be assigned a different set of input output pairs generated from different state-space systems.

```python
import hashlib
import numpy as np

def get_my_params(student_number):
    """AI 321 / EE 298 -- personalized exercise parameters.
    Deterministic, unique per official student number. Run once at the
    top of your notebook/script; use the returned fields wherever an
    exercise below tells you to."""
    digest = hashlib.sha256(str(student_number).strip().encode()).hexdigest()
    seed = int(digest[:8], 16)
    return {
        "student_number": str(student_number).strip(),
        "seed": seed,
        "rng": np.random.default_rng(seed),
        "offset_small": seed % 30,        # e.g. a day/index offset, 0-29
        "offset_large": seed % 3000,      # e.g. a row offset into a long series
        "subset_size": 30 + (seed % 21),  # e.g. 30-50, for "sample N series" tasks
        "choice_of_8": seed % 8,          # e.g. selecting 1 of 8 provided variants
    }
params = get_my_params("2021-XXXXX")  # <-- replace with YOUR official student number
```

A MATLAB equivalent named get_my_params.m is provided alongside this exercise for students working in MATLAB.

2.3 Problem 1: Subspace Identification Methods (SIM)

Subspace Identification Models (SIM). In the file "p_dat.mat" is result of simulating the Newell-Lee Evaporator system model[1]. The data is a cell structure with two sets of input, output pairs. Each set is a 3-input, 3-output system, and the number of data points is 1001, the simulation time is from 0 to 500s. For Python there is a library that allows you to load MATLAB *.mat files: https://pypi.org/project/mat73/.

The two sets of data correspond to: 1) small input amplitudes and 2) large input amplitudes. To verify load the data in the MATLAB workspace and plot the two sets of waveform:

```
>> load p_dat.mat; figure; plot(p_dat{1}); figure; plot(p_dat{2});
```

From this set of data, do the following:

A. Split the data into training and validation. The first 0 to 500 data points is the training set, the 501 to 1001 data points is the validation set.

B. Estimate the state-space model parameters using Subspace Identification Method using Canonical Variate Analysis. Estimate the model parameters using the train data and test the performance on the validation data. In model estimation determine the following:

   a. The best model order according to the metric for inputs small amplitudes on the training data, just to check how well the model performs.

   b. The best model order according to the metric for inputs small amplitudes on the validation data, for all output variables.

   c. The best model order according to the metric for inputs large amplitudes on the training data, just to check how well the model performs.

   d. The best model order according to the metric for inputs large amplitudes on the validation data, for all output variables.

   e. Show the plots of the output and model prediction on the training data and the validation data.

C. Change the model estimation algorithm to Multivariable Output Error State Space or MOESP and compare the performance with CVA, which one performs better?

D. Change the model estimation algorithm to State-Space ARX model or SSARX and compare with the other model estimation methods.

E. What are your insights into the model estimation method using state-space models? Compare the performance of the different modeling algorithms? What can be attributed to the different in model performance if any?

2.4 Problem 2: Unknown State-Space Model

Unknown State-Space Model. A continuous state-space system with unknown order and parameters was used to generate the input and output waveforms given. Use system identification to estimate the parameters of the continuous state-space system from the input and

[1] https://www.mathworks.com/matlabcentral/fileexchange/68641-newell-lee-evaporator-system-for-system-identification

Student	Model Name	Student	Model Name
Asares	s1.mat	Benzon	s2.mat
Bolo	s3.mat	Dayo	s4.mat
De Guia	s5.mat	Francisco	s6.mat
Ignacio	s7.mat	Mendoza	s8.mat
Munar	s9.mat	Renovalles	s10.mat
Rosell	s11.mat	Carcillar	s12.mat

Table 2.1: Student-to-dataset assignment for MP2 (12 students). `s13.mat` is the common validation set.

output waveforms. The following are the inputs used to generate the outputs: step function, impulse, sine wave, rectangular wave, periodic impulse and amplitude modulated random binary sequence (APRBS). The output waveforms are corrupted by a random noise with fixed noise variance to simulate the measurement of imperfect sensors in the data.

a. From the given waveforms, use all the data as the training. A different set of waveforms will be used for validation.

b. Using the data, estimate the model parameters using the training data and test using the validation data using the metrics: MSE, NMSE and R^2.

c. The task is to estimate the parameters of a suitable state-space model using a subspace identification model such as CVA, MODESP, or SSARX. Report about the best model performance using the subspace identification model.

d. Estimate the parameters of the model using any of the prediction error models (PEM) available: AR, ARX, ARMA, ARMAX, Transfer function, Box-Jenkins, etc. Report about the best model performance using the subspace identification model.

e. There are eleven state-space systems assigned below, one per student (`s1.mat` through `s12.mat`). An additional reserved set, `s13.mat`, this is the common blind validation set that everyone will use to benchmark the model performance.

f. **BONUS.** You can use any of the supported machine-learning models to estimate the parameters of the unknown state-space system, if you think the previous SIM and PEM models are insufficient for the task. Why did you choose a particular ML / NN models and report on the performance of the your chosen model against the SIM and PEM methods.

Generating Your Own APRBS Signals

The waveform data you were given already includes one APRBS realization (column 6 of `Input` in your assigned `s#.mat` file). You may want more: to explore how your model performs on excitation it wasn't given. You can generate your own APRBS signals directly from your assigned system model (`sys#.mat`) using the sample code below.

An APRBS (Amplitude-modulated Pseudo-Random Binary Sequence) generalizes a PRBS: instead of switching between just two fixed levels, it holds a random amplitude level for a random duration, then switches to a new random level, and so on. A *small-amplitude* APRBS gently perturbs the system near a small operating range; a *large-amplitude* APRBS excites it more aggressively across a wider range.

Python

```python
import numpy as np

def generate_aprbs(N, hold_range, amp_range, rng=None):
    """Amplitude-modulated Pseudo-Random Binary Sequence (APRBS)."""
    if rng is None:
        rng = np.random.default_rng()
    u = np.zeros(N)
    idx = 0
    while idx < N:
        seglen = int(rng.integers(hold_range[0], hold_range[1] + 1))
        amp = rng.uniform(amp_range[0], amp_range[1])
        seg_end = min(idx + seglen, N)
        u[idx:seg_end] = amp
        idx = seg_end
    return u

u_small = generate_aprbs(2001, (20, 150), (-0.2, 0.2))   # gentle
u_large = generate_aprbs(2001, (20, 150), (-1.0, 1.0))   # aggressive
```

A MATLAB equivalent, `generate_aprbs.m`, is provided alongside this exercise (as is a matching `generate_aprbs.py` for the Python version above):

MATLAB

```matlab
u_small = generate_aprbs(2001, [20 150], [-0.2 0.2]);   % gentle
u_large = generate_aprbs(2001, [20 150], [-1.0 1.0]);   % aggressive

s = load('sys1.mat'); sys = s.sys1;   % replace sys1/1 with your assigned index
dt = 0.05; t = (0:2000)'*dt;
y_small = lsim(sys, u_small, t);
y_large = lsim(sys, u_large, t);
```

Both functions take a hold-time range (segment length, in samples) and an amplitude range as arguments, so you can tune how quickly the signal switches levels and how large those levels are. The example amplitude ranges above (± 0.2 for small, ± 1.0 for large) match the scale already used to build your assigned dataset's own APRBS column, so the "large" example is directly comparable to what you've already been given, and "small" gives you a gentler counterpart to compare against.

Reflection and Academic Integrity Declaration

This section is required and graded on the depth and specificity of your answers, not on whether or how much you used AI. This responses in this section can be considered when grading the exercise as a whole.

AI 321's policy on AI tools — transparency, not prohibition

You are free to use AI tools (Claude, ChatGPT, Copilot, etc) to help you learn, debug, and understand concepts in this course. What you can not do is submit AI-generated code, derivations, or written answers as your own work without disclosing them and demonstrating you understand them. Doing so is academic dishonesty, equivalent to plagiarism, whether or not the code runs correctly.

1. Approach Summary

In 3-5 sentences, describe what you did and why, in your own words, as if explaining it to a classmate who has not seen this exercise.

[*Response*]

2. Key Insight

What is one result or observation from this exercise that surprised you, or didn't match your initial intuition?

[*student response*]

3. Most Difficult Part

What part of this exercise took the longest to get right, and how did you resolve it?

[*student response*]

4. Disclosure on the use of AI systems

List every part of this exercise where you used an AI tool, one row per use. Write "None" in the first row if you did not use AI at all. The last column is the part that can be considered in grading this exercise. Be clear on your statements, on how AI actually helped you if it did. They do make mistakes, you can also list here the instances where the AI failed and you were able to recognize its failure.

Section / task	**Tool used**	**What you asked it for**	**Explain the result in your own words, as if teaching it**
e.g., Problem 2, debugging	*e.g., ChatGPT*	*e.g., fix a shape mismatch in my autocorrelation matrix*	[*student response*]

5. Self-Assessed Understanding

For each objective or major problem in this exercise, rate your confidence (Confident / Mostly confident / Still shaky) and add one sentence why.

Objective / problem	**Confidence**	**Why (one sentence)**

6. Academic Honor Pledge

By submitting this exercise, I hereby affirm upon my honor the following:

1. The analysis, results, codes, answers to questions, discussions and conclusions in this machine exercise reflect my own understanding and work.
2. I used my own official student number whenever it is required, and the results generated were computed from my own personalized parameters/data, and not from anyone else.
3. I have disclosed fully and accurately any use of AI tools, LLMs, AI agents or similar systems in Section 4 above, and I can explain, in my own words, every part of what I submitted if asked.
4. I understand that submitting AI-generated work as my own can be considered as plagiarism and is an academic dishonesty, and that I abide with the University's policy on Academic Integrity as written in the Student's Code of Conduct.

Signed (name): _____ **Student number:** _____

Date: _____